Module 2 — Workflows & Best Practices

Lesson 2.2

TDD with Claude

Write the failing test yourself, then have Claude implement to pass. The single most reliable workflow for correctness.

Why it matters

LLMs are eager to please. Given a vague spec ("add a function that parses dates"), they'll produce something that compiles and looks right but misses edge cases. A failing test pins down exact behavior. Claude can't satisfy it by being eloquent; it has to make it pass.

The workflow

```
1. YOU:    Write a failing test that describes the behavior
2. YOU:    Confirm it actually fails (run it)
3. YOU:    Ask Claude: "Make this test pass. Don't modify the test."
4. CLAUDE: Implements; runs test; iterates until green
5. YOU:    Add the next failing test (edge case, error case)
6. Repeat until you've covered the contract
```

Why "don't modify the test" matters

Without that instruction, Claude will sometimes "fix" the test to match a flawed implementation. It's not malice — the prompt was ambiguous about which is ground truth. Be explicit.

A stronger variant: put the test in a file with a comment header:

```
# DO NOT MODIFY THIS TEST — it is the spec for the implementation.
def test_parse_iso_date_with_offset():
    assert parse_iso_date("2026-05-26T10:00:00+05:30") == ...
```

What a good Claude-driven TDD prompt looks like

> *"The test* `tests/test_dates.py::test_parse_iso_date_with_offset` *is failing. Implement* `src/dates.py::parse_iso_date` *to make it pass. Don't modify the test or any other test. Run* `pytest tests/test_dates.py -k parse_iso_date` *to verify."*

This prompt does four things right:

1. Names the failing test precisely
2. Names the function to implement
3. Forbids modifying the spec
4. Names the verification command

When TDD shines

- **New pure functions** — parsers, serializers, calculations
- **API contract changes** — write integration tests for the new shape first
- **Bug fixes** — write a test that reproduces the bug, then ask Claude to fix it
- **Refactors** — pin behavior with tests, then refactor; tests should still pass

When TDD is awkward

- **UI changes** — visual correctness is hard to test mechanically; use verify instead
- **Exploration spikes** — when you don't yet know the contract, you can't write the test

- **Glue code** — wiring four libraries together is mostly verified by running, not unit testing

The bug-fix variant (regression-driven)

For bugs, the discipline is:

1. Reproduce in a test (you, not Claude — your reproduction is the spec)
2. Confirm the test fails
3. Claude fixes the underlying issue
4. Test passes
5. Commit the test alongside the fix

The test then permanently guards against regression. This is the highest-ROI testing you can do.

Try it

1. Pick a small utility function in your codebase that lacks tests.
2. Write 3–5 failing tests that capture its current behavior (yes, even though it's "working" — these are your safety net).
3. Hand it to Claude: "Refactor this for clarity. All tests must still pass; don't change them."
4. Observe whether Claude's refactor breaks anything you didn't anticipate.

Gotchas

- **Don't let Claude generate the tests.** It will test what's easy to test, not what matters. Tests are *your* spec.
- **Make sure the test actually fails first.** A test that's accidentally already passing tells you nothing.
- **Watch for "passes by mocking everything"** — if Claude's implementation passes the test by mocking the thing the test was supposed to verify, push back. Mock at I/O boundaries, not at the logic under test.
- **Integration > unit for high-leverage tests.** A unit test on a mocked DB doesn't prove the migration works. Hit the real thing in CI.

Further reading

- 2.4 Debugging patterns — the bug-fix variant in depth
- 2.6 Delegation — when to delegate test-writing vs. implementation

Next

→ 2.3 Review and refactor